

```
#!/usr/bin/env python3
```

```
"""
```

VSTCosmos v90 – Protocolo cíclico: 100 ciclos de 13 fases

Base: v88 exacto.

Cambios respecto a v88:

1. Signo canónico del gradiente: positivo = fuente a la derecha (+60°)

Una línea: (E_R - E_L) en lugar de (E_L - E_R)

2. Protocolo extendido: 100 ciclos completos de 13 fases

El campo persiste entre ciclos – sin reinicio

El audio avanza secuencialmente entre ciclos (punteros por clave)

Log detallado solo en ciclos 1, 10, 25, 50, 75, 100

Sin nuevos parámetros. Sin nuevas restricciones. El sistema funciona – le damos t

```
"""
```

```
import numpy as np
```

```
import scipy.io.wavfile as wav
```

```
import matplotlib.pyplot as plt
```

```
import csv
```

```
import warnings
```

```
import os
```

```
from scipy import signal as scipy_signal
```

```
try:
```

```
    import soundfile as sf
```

```
    HAS_SF = True
```

```
except ImportError:
```

```
    HAS_SF = False
```

```
warnings.filterwarnings('ignore')
```

```
# =====
```

```
# PARÁMETROS DE LA FÍSICA DEL CAMPO
```

```
# =====
```

```
DIM_INTERNA = 32
```

```
DIFUSION_BASE      = 0.15
```

```
GANANCIA_REACCION  = 0.05
```

```
OMEGA_MIN          = 0.05
```

```
OMEGA_MAX          = 0.50
```

```
AMORT_MIN          = 0.01
```

```
AMORT_MAX          = 0.08
```

```
PHI_EQUILIBRIO     = 0.5
```

```

VENTANA_FFT_MS = 25
HOP_FFT_MS     = 10
F_MIN          = 80
F_MAX          = 8000

T_PROFUNDA_SEG = 1.0 / OMEGA_MIN
T_RECIENTE_SEG = 1.0 / OMEGA_MAX
T_PROFUNDA_PASOS = int(T_PROFUNDA_SEG / 0.01)
T_RECIENTE_PASOS = int(T_RECIENTE_SEG / 0.01)

ETA_PROFUNDA_BASE = (1.0 / T_PROFUNDA_PASOS) / DIFUSION_BASE
ETA_RECIENTE_BASE = (1.0 / T_RECIENTE_PASOS) / DIFUSION_BASE
TAU_PROFUNDA      = OMEGA_MIN
TAU_RECIENTE      = OMEGA_MIN * 0.5
TAU_EFICIENCIA    = int(1.0 / (OMEGA_MIN * 0.01))
TAU_EXPLORACION   = int(T_RECIENTE_SEG / 0.01)

LIMITE_MIN = 0.0
LIMITE_MAX = 1.0
W_MAX      = 1.0
ALPHA_FIJO = 0.05
DT         = 0.01
DIM_TIME   = 100

# Constantes físicas para binaural
DIAMETRO_CABEZA = 0.175
VELOCIDAD_SONIDO = 343.0
ITD_MAX_SEG      = DIAMETRO_CABEZA / VELOCIDAD_SONIDO
F_TRANS_HZ       = VELOCIDAD_SONIDO / DIAMETRO_CABEZA # ≈ 1960 Hz

# =====
# ARQUITECTURA DEL CAMPO EXPANDIDO
# =====
DIM_GANGLIO = DIM_INTERNA // 2 # 16
DIM_AUD     = DIM_GANGLIO      # 16 ← definido UNA SOLA VEZ, antes de BANDA_T
DIM_ACT     = DIM_GANGLIO // 2 # 8

DIM_AUD_L   = DIM_AUD
DIM_AUD_R   = DIM_AUD
DIM_ACT_PERM = DIM_ACT
DIM_ACT_GEOM = DIM_ACT
DIM_ACT_BUSC = DIM_ACT
DIM_ACT_MANT = DIM_ACT

# BANDA_TRANS: después de DIM_AUD
BANDA_TRANS = int(DIM_AUD * np.log10(F_TRANS_HZ / F_MIN))

```

```

        / np.log10(F_MAX / F_MIN))
BANDA_TRANS = max(1, min(BANDA_TRANS, DIM_AUD - 1))    # = 11

# Parámetros para orientación
K_BUSC          = T_PROFUNDA_SEG / T_RECIENTE_SEG    # = 10
K_ORIENT        = T_PROFUNDA_SEG / T_RECIENTE_SEG    # = 10
DECAIMIENTO_ACT_BUSC = DT / T_RECIENTE_SEG          # = 0.005
EPSILON_BUSC_G   = DIFUSION_BASE * K_BUSC * DT       # = 0.015

# Índices de regiones
idx = {}
idx['int']       = (0,                                DIM_INTERNA)
idx['G']         = (DIM_INTERNA,                      DIM_INTERNA + DIM_GANGLIO)
idx['aud_L']     = (idx['G'][1],                      idx['G'][1] + DIM_AUD_L)
idx['aud_R']     = (idx['aud_L'][1],                  idx['aud_L'][1] + DIM_AUD_R)
idx['act_perm']  = (idx['aud_R'][1],                  idx['aud_R'][1] + DIM_ACT_PERM)
idx['act_geom']  = (idx['act_perm'][1],                idx['act_perm'][1] + DIM_ACT_GEOM)
idx['act_busc']  = (idx['act_geom'][1],                idx['act_geom'][1] + DIM_ACT_BUSC)
idx['act_mant']  = (idx['act_busc'][1],                idx['act_busc'][1] + DIM_ACT_MANT)
DIM_TOTAL = idx['act_mant'][1]

# Topología - act_busc fuera del equilibrio difusivo (desde v86)
VECINDADES = [
    ('int',      'G'),
    ('G',        'aud_L'),
    ('G',        'aud_R'),
    ('G',        'act_perm'),
    ('G',        'act_geom'),
    # ('G',        'act_busc'), ← ELIMINADA desde v86
    ('G',        'act_mant'),
    ('aud_L',    'aud_R'),
    ('act_perm', 'aud_L'),
    ('act_perm', 'aud_R'),
    ('act_geom', 'aud_L'),
    ('act_geom', 'aud_R'),
]

print("=" * 100)
print("VSTCosmos v90 - Protocolo cíclico: 100 ciclos de 13 fases")
print("")
print("  Base: v88. Cambios:")
print("  1. Signo canónico del gradiente (positivo = derecha)")
print("  2. 100 ciclos completos de 13 fases - el campo persiste entre ciclos")
print("")
print(f"  BANDA_TRANS={BANDA_TRANS} (F>{F_TRANS_HZ:.0f}Hz, {DIM_AUD-BANDA_TRANS})")
print(f"  K_BUSC={K_BUSC}, K_ORIENT={K_ORIENT}, EPSILON_BUSC_G={EPSILON_BUSC_G}")
print(f"  DIM_TOTAL={DIM_TOTAL}")

```

```

print("=" * 100)

# =====
# CARGA DE ARCHIVOS BINAURALES
# =====

def cargar_todos_binaurales(directorio='audio_binaural', duracion=35.0):
    mapping = {
        'voz_pos':      'Voz_Estudio_pos60deg.wav',
        'voz_neg':      'Voz_Estudio_neg60deg.wav',
        'musica_pos':    'Brandemburgo_pos60deg.wav',
        'voz_viento1_pos': 'Voz+Viento_1_pos60deg.wav',
        'voz_viento2_pos': 'Voz+Viento_2_pos60deg.wav',
        'tono_pos':      'Tono puro_pos60deg.wav',
        'ruido_pos':     'Ruido blanco_pos60deg.wav',
        'ritmos_pos':    'Ritmos aleatorios_pos60deg.wav',
        'ondas_pos':     'Ondas mixtas_pos60deg.wav',
        'pulso_pos':     'Pulso logaritmico_pos60deg.wav',
        'viento_pos':    'Viento_pos60deg.wav',
        'bigbang_pos':   'BigBang_pos60deg.wav',
    }
    archivos = {}
    print(f"\n[Carga] Desde '{directorio}/'...'")
    for clave, filename in mapping.items():
        filepath = os.path.join(directorio, filename)
        if not os.path.exists(filepath):
            print(f"⚠ {clave:22s} no encontrado")
            continue
        try:
            data, sr = sf.read(filepath, dtype='float32')
            if clave != 'bigbang_pos' and duracion is not None:
                n = int(sr * duracion)
                if data.ndim == 1:
                    data = data[:n]
                    if len(data) < n:
                        data = np.pad(data, (0, n - len(data)))
                else:
                    data = data[:n, :]
                    if data.shape[0] < n:
                        data = np.pad(data, ((0, n - data.shape[0]), (0, 0)))
            if data.ndim == 1:
                canal_L = data
                canal_R = data.copy()
            else:
                canal_L = data[:, 0]
                canal_R = data[:, 1] if data.shape[1] > 1 else data[:, 0].copy()
            dur_real = len(canal_L) / sr
            archivos[clave] = (filepath, sr, canal_L, canal_R)

```

```

        print(f"      ✓ {clave:22s} {filename} ({dur_real:.1f}s)")
    except Exception as e:
        print(f"      ✗ {clave:22s} {e}")
    print(f"  Carga completada: {len(archivos)} archivos.")
    return archivos

# =====
# CLASE EXPLORADOR
# =====
class ExploradorActuadores:
    def __init__(self):
        self.historial      = []
        self.mejor_config   = None
        self.mejor_eficiencia = 0.0
        self.pasos_en_lf    = 0

    def actualizar(self, lf_activa, efic, fL, fR, sesgo):
        if lf_activa:
            self.pasos_en_lf += 1
            self.historial.append((fL, fR, sesgo, efic))
            if efic > self.mejor_eficiencia:
                self.mejor_eficiencia = efic
                self.mejor_config = (fL, fR, sesgo)
        else:
            self.pasos_en_lf = 0

# =====
# FUNCIONES BASE
# =====
def inicializar_campo():
    np.random.seed(None)
    Phi_total      = np.random.normal(PHI_EQUILIBRIO, 0.01, (DIM_TOTAL, DIM_TIME))
    Phi_vel_total  = np.zeros((DIM_TOTAL, DIM_TIME))
    return Phi_total, Phi_vel_total

def inicializar_memorias():
    W_prof          = np.zeros((DIM_INTERNA, DIM_AUD))
    W_rec           = np.zeros((DIM_INTERNA, DIM_AUD))
    Phi_int_historia = np.zeros((DIM_INTERNA, DIM_TIME))
    return W_prof, W_rec, Phi_int_historia

def _perfil_espectral_region(region, dim):
    n_bins = 50
    perfil = np.zeros(n_bins)
    for banda in range(min(dim, region.shape[0])):
        serie      = region[banda, :] - np.mean(region[banda, :])
        fft         = np.fft.rfft(serie)

```

```

        perfil += np.abs(fft)[:n_bins] ** 2
    return perfil / max(1, dim)

def calcular_ged_entre(region_a, region_b):
    p_a = _perfil_espectral_region(region_a, region_a.shape[0])
    p_b = _perfil_espectral_region(region_b, region_b.shape[0])
    return float(np.mean(np.abs(p_a - p_b)))

def calcular_frecuencias_naturales(dim):
    bandas = np.arange(dim)
    t = np.log1p(bandas) / np.log1p(max(dim - 1, 1))
    omega = OMEGA_MIN + (OMEGA_MAX - OMEGA_MIN) * t
    amort = AMORT_MIN + (AMORT_MAX - AMORT_MIN) * t
    return omega.reshape(-1, 1), amort.reshape(-1, 1)

def calcular_promedio_vecinos(Phi_total):
    promedio = np.zeros_like(Phi_total)
    conteo = np.zeros(DIM_TOTAL)
    for reg_a, reg_b in VECINDADES:
        ia0, ia1 = idx[reg_a]
        ib0, ib1 = idx[reg_b]
        n = min(ia1 - ia0, ib1 - ib0)
        for d in range(n):
            if ia0 + d < DIM_TOTAL and ib0 + d < DIM_TOTAL:
                promedio[ia0 + d, :] += Phi_total[ib0 + d, :]
                promedio[ib0 + d, :] += Phi_total[ia0 + d, :]
                conteo[ia0 + d] += 1
                conteo[ib0 + d] += 1
    for i in range(DIM_TOTAL):
        if conteo[i] > 0:
            promedio[i, :] /= conteo[i]
        else:
            promedio[i, :] = Phi_total[i, :]
    return promedio

# =====
# PREPARAR OBJETIVO
# =====
def preparar_objetivo_canal(canal, sr, idx_paso, ventana_muestras,
                            hop_muestras, dim_aud, dim_time):
    inicio = idx_paso * hop_muestras
    fin = inicio + ventana_muestras
    segmento = canal[inicio:fin] if fin <= len(canal) else canal[inicio:]
    if len(segmento) < ventana_muestras:
        segmento = np.pad(segmento, (0, ventana_muestras - len(segmento)))

    fft = np.fft.rfft(segmento)

```

```

potencia = np.abs(fft) ** 2
freqs    = np.fft.rfftfreq(len(segmento), 1 / sr)

bandas   = np.logspace(np.log10(F_MIN), np.log10(F_MAX), dim_aud + 1)
objetivo = np.zeros(dim_aud)
for b in range(dim_aud):
    mask = (freqs >= bandas[b]) & (freqs < bandas[b + 1])
    if np.any(mask):
        objetivo[b] = np.mean(potencia[mask])

max_val = np.max(objetivo)
if max_val > 0:
    objetivo /= max_val

return objetivo.reshape(-1, 1) * np.ones((1, dim_time))

# =====
# GRADIENTE ENERGÉTICO DIRIGIDO (variable primaria de orientación)
# =====
def calcular_gradiente_energetico_dirigido(obj_L, obj_R):
    """
    Diferencia de energía espectral entre canales L y R
    solo en las bandas altas donde existe ILD binaural.

    gradiente  $\in$  (-1, 1) por construcción – sin clip externo.

    Convención canónica:
        positivo → fuente a la DERECHA (+60°)
        negativo → fuente a la IZQUIERDA (-60°)

    El preprocesador asigna canal_L=sombreado para +60°,
    por lo tanto R tiene más energía en altas para +60°:
        gradiente = (E_R - E_L) / total → positivo para +60°
    """
    if BANDA_TRANS >= DIM_AUD:
        return 0.0
    energia_L = float(np.mean(obj_L[BANDA_TRANS:, :] ** 2))
    energia_R = float(np.mean(obj_R[BANDA_TRANS:, :] ** 2))
    total     = energia_L + energia_R + 1e-10
    return (energia_R - energia_L) / total

# =====
# COHERENCIA (solo diagnóstico – no driver)
# =====
def calcular_coherencia_dirigida(obj_L, obj_R, W_prof, region_int):
    """
    Coherencia en bandas altas. Solo para diagnóstico en v88.

```

```

"""
if BANDA_TRANS >= DIM_AUD:
    return 0.0, 0.0, 0.0
n_prof = W_prof.shape[0]
n_cols = W_prof.shape[1]
n_int = region_int.shape[0]
n_altas = DIM_AUD - BANDA_TRANS
if n_altas <= 0:
    return 0.0, 0.0, 0.0
perfil_L = obj_L[BANDA_TRANS:, :].mean(axis=1)
perfil_R = obj_R[BANDA_TRANS:, :].mean(axis=1)
perfil_i = region_int.mean(axis=1)
min_c = min(n_cols - BANDA_TRANS, n_altas)
min_p = min(n_prof, n_int)
if min_c <= 0 or min_p <= 0:
    return 0.0, 0.0, 0.0
W_alto = W_prof[:min_p, BANDA_TRANS:BANDA_TRANS + min_c]
pred_L = W_alto @ perfil_L[:min_c].reshape(-1, 1)
pred_R = W_alto @ perfil_R[:min_c].reshape(-1, 1)
ref = perfil_i[:min_p].reshape(-1, 1)
err_L = float(np.mean((pred_L - ref) ** 2))
err_R = float(np.mean((pred_R - ref) ** 2))
total = err_L + err_R + 1e-10
return float((err_R - err_L) / total), err_L, err_R

# =====
# ACT_BUSC - respuesta rápida al gradiente energético
# =====
def actualizar_act_busc_desde_gradiente(Phi_total, gradiente_E, dt):
    """
    act_busc recibe el gradiente energético. Respuesta rápida (T_RECIENTE).
    Señal centrada en PHI_EQUILIBRIO para signo informativo.

    Con gradiente=+0.2: señal =  $0.5 + \tanh(10 \times 0.2) \times 0.15 \approx 0.649$ 
    Con gradiente=-0.2: señal  $\approx 0.351$ 
    act_busc centrado:  $\pm 0.149$  - comparable con  $\Phi_G$ 
    """
    ab0, ab1 = idx['act_busc']
    señal = PHI_EQUILIBRIO + float(np.tanh(K_BUSC * gradiente_E)) * DIFUSION_BASE
    Phi_total[ab0:ab1, :] = (
        (1.0 - DECAIMIENTO_ACT_BUSC) * Phi_total[ab0:ab1, :] +
        DECAIMIENTO_ACT_BUSC * señal
    )
    return Phi_total

# =====
# FORZAMIENTO DIRIGIDO act_busc → G

```

```

# =====
def aplicar_forzamiento_busc_a_ganglio(Phi_total, dt):
    """Acoplamiento dirigido – propaga gradiente desde act_busc a  $\Phi_G$ ."""
    ab0, ab1 = idx['act_busc']
    g0, g1 = idx['G']
    estado_busc = float(np.mean(Phi_total[ab0:ab1, :])) - PHI_EQUILIBRIO
    n = min(ab1 - ab0, g1 - g0)
    Phi_total[g0:g0 + n, :] += EPSILON_BUSC_G * estado_busc
    return Phi_total

# =====
# ACT_GEOM – orientación estable por el mismo gradiente
# =====
def aplicar_orientacion_por_gradiente(Phi_total, gradiente_E, dt):
    """
    act_geom recibe la misma señal que act_busc – gradiente energético.
    Respuesta lenta: K_ORIENT × dt acumula gradiente en escala temporal larga.

    Una sola variable de orientación, dos escalas temporales:
    act_busc = respuesta rápida (decaimiento T_RECIENTE)
    act_geom = integración lenta (K_ORIENT × DIFUSION_BASE × dt por paso)
    """
    acg0 = idx['act_geom'][0]
    acg1 = idx['act_geom'][1]
    mitad = max(1, (acg1 - acg0) // 2)
    señal = float(np.clip(
        gradiente_E * DIFUSION_BASE * K_ORIENT * dt, -0.1, 0.1
    ))
    Phi_total[acg0:acg0 + mitad, :] += señal
    Phi_total[acg0 + mitad:acg1, :] -= señal
    return Phi_total

# =====
# ACTUACIÓN CUALITATIVA
# =====
def calcular_parametros_actuacion(Phi_total):
    act_perm = Phi_total[idx['act_perm'][0]:idx['act_perm'][1], :]
    act_geom = Phi_total[idx['act_geom'][0]:idx['act_geom'][1], :]
    nivel_perm = float(np.mean(np.tanh(act_perm)))
    frac_base = 0.25 + 0.75 * (nivel_perm + 1.0) / 2.0
    mitad = max(1, DIM_ACT // 2)
    g_baja = float(np.mean(act_geom[:mitad, :]))
    g_alta = float(np.mean(act_geom[mitad:, :]))
    sesgo = float(np.tanh(g_alta - g_baja))
    asimetria = float(np.tanh(g_baja - g_alta))
    frac_L = float(np.clip(frac_base * (1.0 + asimetria * 0.5), 0.1, 1.0))
    frac_R = float(np.clip(frac_base * (1.0 - asimetria * 0.5), 0.1, 1.0))

```

```

    return frac_L, frac_R, sesgo, asimetria, nivel_perm

def aplicar_entrada_cualitativa(Phi_total, obj_L, obj_R, frac_L, frac_R, sesgo):
    def aplicar_canal(obj_full, frac, sl):
        n_act = max(1, int(DIM_AUD * frac))
        if sesgo > 0:
            ini = int(DIM_AUD * min(sesgo, 0.8) * 0.5)
            fin = min(DIM_AUD, ini + n_act)
        else:
            ini, fin = 0, n_act
        obj_mod = np.zeros((DIM_AUD, DIM_TIME), dtype=np.float32)
        obj_mod[ini:fin, :] = obj_full[ini:fin, :]
        Phi_total[sl, :] = ((1 - ALPHA_FIJO) * Phi_total[sl, :]
                             + ALPHA_FIJO * obj_mod)
        aplicar_canal(obj_L, frac_L, slice(idx['aud_L'][0], idx['aud_L'][1]))
        aplicar_canal(obj_R, frac_R, slice(idx['aud_R'][0], idx['aud_R'][1]))
    return Phi_total

# =====
# EXPLORACIÓN ACTIVA
# =====
def explorar_actuadores(Phi_total, explorador, lf_activa, eficiencia, dt):
    AMPLITUD_MAX = DIFUSION_BASE
    ap0, ap1 = idx['act_perm']
    ag0, ag1 = idx['act_geom']
    if lf_activa:
        amplitud = AMPLITUD_MAX * min(1.0, explorador.pasos_en_lf / TAU_EXPLORACION)
        if explorador.mejor_config is not None:
            nivel = float(np.mean(np.tanh(Phi_total[ap0:ap1, :])))
            sesgo = ((explorador.mejor_config[0] + explorador.mejor_config[1])
                     / 2.0 - nivel)
            ruido_perm = np.random.normal(sesgo * 0.5, amplitud,
                                           (ap1 - ap0, DIM_TIME))
        else:
            ruido_perm = np.random.normal(0, amplitud, (ap1 - ap0, DIM_TIME))
            ruido_geom = np.random.normal(0, amplitud * 0.5, (ag1 - ag0, DIM_TIME))
            Phi_total[ap0:ap1, :] += ruido_perm * dt
            Phi_total[ag0:ag1, :] += ruido_geom * dt
    else:
        if explorador.mejor_config is not None:
            nivel = float(np.mean(np.tanh(Phi_total[ap0:ap1, :])))
            corr = (explorador.mejor_config[0] - nivel) * DIFUSION_BASE * dt
            Phi_total[ap0:ap1, :] += corr
    return Phi_total

# =====
# PLASTICIDAD DUAL

```

```

# =====
def aplicar_plasticidad_dual(region_int, region_aud, W_prof, W_rec,
                             Phi_int_historia, dt):
    min_prof = min(W_prof.shape[0], region_int.shape[0])
    min_cols = min(W_prof.shape[1], region_aud.shape[0])
    W_p = W_prof[:min_prof, :min_cols]
    W_r = W_rec[:min_prof, :min_cols]
    r_i = region_int[:min_prof, :]
    r_a = region_aud[:min_cols, :]
    corr_prof = (r_i @ r_a.T) / DIM_TIME
    dW_prof = ETA_PROFUNDA_BASE * corr_prof - TAU_PROFUNDA * W_p
    W_p_nueva = np.clip(W_p + dW_prof * dt, -W_MAX, W_MAX)
    W_prof_nueva = W_prof.copy()
    W_prof_nueva[:min_prof, :min_cols] = W_p_nueva
    pred_rec = np.tanh(W_r @ r_a)
    error_rec = float(np.mean((pred_rec - r_i) ** 2))
    pred_prof = W_p_nueva @ r_a
    error_prof = float(np.mean((pred_prof - r_i) ** 2))
    coherencia = error_prof / (error_rec + error_prof + 1e-10)
    tasa_aprendizaje = ETA_RECIENTE_BASE * coherencia
    corr_rec = (r_i @ r_a.T) / DIM_TIME
    dW_rec = tasa_aprendizaje * corr_rec - TAU_RECIENTE * W_r
    W_r_nueva = np.clip(W_r + dW_rec * dt, -W_MAX, W_MAX)
    W_rec_nueva = W_rec.copy()
    W_rec_nueva[:min_prof, :min_cols] = W_r_nueva
    M_plast = np.zeros((DIM_INTERNA, DIM_TIME))
    delta_p = W_p_nueva @ r_a - r_i
    delta_r = W_r_nueva @ r_a - r_i
    M_plast[:min_prof, :] = (delta_p + delta_r) * 0.01
    Phi_int_historia_nueva = 0.95 * Phi_int_historia + 0.05 * region_int
    return (W_prof_nueva, W_rec_nueva, M_plast,
            error_rec, coherencia, Phi_int_historia_nueva)

# =====
# EFICIENCIA Y MÉTRICAS
# =====
def calcular_eficiencia(Phi_total, ged_actual):
    region_int = Phi_total[idx['int'][0]:idx['int'][1], :]
    variacion = float(np.mean(np.abs(np.diff(region_int, axis=1))))
    return ged_actual / (variacion + 1e-10), variacion

def calcular_senal_busqueda(Phi_total):
    ab0, ab1 = idx['act_busc']
    return float(np.mean(Phi_total[ab0:ab1, :])) - PHI_EQUILIBRIO

# =====
# ACTUALIZACIÓN PRINCIPAL DEL CAMPO

```

```

# =====
def actualizar_campo(Phi_total, Phi_vel_total, W_prof, W_rec,
                    Phi_int_historia, obj_L, obj_R,
                    frac_L, frac_R, sesgo, dt):

    omega_n, amort_n = calcular_frecuencias_naturales(DIM_TOTAL)
    prom      = calcular_promedio_vecinos(Phi_total)
    difusion = DIFUSION_BASE * (prom - Phi_total)
    desv      = Phi_total - prom
    reaccion  = GANANCIA_REACCION * desv * (1 - desv ** 2)
    term_osc  = (-omega_n ** 2 * (Phi_total - PHI_EQUILIBRIO)
                - amort_n * Phi_vel_total)

    region_int = Phi_total[idx['int'][0]:idx['int'][1], :]
    aud_L      = Phi_total[idx['aud_L'][0]:idx['aud_L'][1], :]
    aud_R      = Phi_total[idx['aud_R'][0]:idx['aud_R'][1], :]
    aud_comb   = (aud_L + aud_R) / 2.0

    W_prof, W_rec, M_plast, error_rec, coherencia, Phi_int_historia = \
        aplicar_plasticidad_dual(
            region_int, aud_comb, W_prof, W_rec, Phi_int_historia, dt
        )

    M_campo = np.zeros_like(Phi_total)
    n_m     = M_plast.shape[0]
    M_campo[idx['int'][0]:idx['int'][1] + n_m, :] = M_plast

    Phi_total = aplicar_entrada_cualitativa(Phi_total, obj_L, obj_R,
                                             frac_L, frac_R, sesgo)

    dPhi_vel = term_osc + reaccion + difusion + M_campo
    Phi_vel_n = Phi_vel_total + dt * dPhi_vel
    Phi_nueva = Phi_total + dt * Phi_vel_n

    var_int = np.var(Phi_nueva[idx['int'][0]:idx['int'][1], :])
    if var_int < DIFUSION_BASE * 1e-4:
        Phi_nueva[idx['int'][0]:idx['int'][1], :] += \
            np.random.normal(0, 0.01, (DIM_INTERNA, DIM_TIME))

    lf_activa = error_rec > DIFUSION_BASE ** 2

    return (np.clip(Phi_nueva, LIMITE_MIN, LIMITE_MAX),
            np.clip(Phi_vel_n, -5.0, 5.0),
            W_prof, W_rec, Phi_int_historia,
            lf_activa, error_rec, coherencia)

# =====

```

```

# ENTRENAMIENTO
# =====
def entrenar(archivos, duracion=30.0):
    print(f"\n[Fase 1] Entrenamiento (voz +60°, {duracion}s)")
    Phi_total, Phi_vel_total = inicializar_campo()
    W_prof, W_rec, Phi_int_historia = inicializar_memorias()
    explorador = ExploradorActuadores()

    _, sr, c_L, c_R = archivos['voz_pos']
    vent = int(sr * VENTANA_FFT_MS / 1000)
    hop = int(sr * HOP_FFT_MS / 1000)
    n_pasos = int(duracion / DT)
    errores = []

    for paso in range(n_pasos):
        obj_L = preparar_objetivo_canal(c_L, sr, paso, vent, hop, DIM_AUD, DIM_TI)
        obj_R = preparar_objetivo_canal(c_R, sr, paso, vent, hop, DIM_AUD, DIM_TI)

        gradiente_E = calcular_gradiente_energetico_dirigido(obj_L, obj_R)
        Phi_total = actualizar_act_busc_desde_gradiente(Phi_total, gradiente_E,
        Phi_total = aplicar_forzamiento_busc_a_ganglio(Phi_total, DT)
        Phi_total = aplicar_orientacion_por_gradiente(Phi_total, gradiente_E, D

        fL, fR, sf_v, _, _ = calcular_parametros_actuacion(Phi_total)

        Phi_total, Phi_vel_total, W_prof, W_rec, Phi_int_historia, \
        _, error_rec, _ = actualizar_campo(
            Phi_total, Phi_vel_total, W_prof, W_rec,
            Phi_int_historia, obj_L, obj_R, fL, fR, sf_v, DT
        )
        errores.append(error_rec)

        if paso % 500 == 0:
            print(f" Paso {paso}/{n_pasos}, error={error_rec:.6f}")

    print(f" ERROR_EQUILIBRIO: {min(errores):.6f}")
    print(f" W_prof: {np.mean(np.abs(W_prof)):.4f}")
    print(f" W_rec: {np.mean(np.abs(W_rec)):.4f}")

    return Phi_total, Phi_vel_total, W_prof, W_rec, Phi_int_historia, explorador

# =====
# SIMULACIÓN DE FASE
# =====
def simular_fase(Phi_total, Phi_vel_total, W_prof, W_rec,
                 Phi_int_historia, historial_ef, explorador,
                 sr, canal_L, canal_R, duracion, verbose=True):

```

```

vent    = int(sr * VENTANA_FFT_MS / 1000)
hop      = int(sr * HOP_FFT_MS / 1000)

if duracion is None:
    n_pasos_max = len(canal_L) // hop
else:
    n_pasos_max = int(duracion / DT)

n_pasos = min(n_pasos_max, 5000)

hist = {k: [] for k in [
    'ged_L', 'ged_R', 'grad_E', 'act_busc', 'coh_rel',
    'geom', 'frac_L', 'frac_R', 'efic', 'lf',
    'w_rec', 'w_prof', 'G_act'
]}

lf_prev = False
for paso in range(n_pasos):
    obj_L = preparar_objetivo_canal(canal_L, sr, paso, vent, hop, DIM_AUD, DI
    obj_R = preparar_objetivo_canal(canal_R, sr, paso, vent, hop, DIM_AUD, DI

    # 1. Gradiente energético (variable primaria de orientación)
    gradiente_E = calcular_gradiente_energetico_dirigido(obj_L, obj_R)

    # 2. Coherencia (solo diagnóstico)
    region_int = Phi_total[idx['int'][0]:idx['int'][1], :]
    coh_rel, _, _ = calcular_coherencia_dirigida(
        obj_L, obj_R, W_prof, region_int
    )

    # 3. act_busc: respuesta rápida
    Phi_total = actualizar_act_busc_desde_gradiente(Phi_total, gradiente_E, D

    # 4. Forzamiento act_busc → G
    Phi_total = aplicar_forzamiento_busc_a_ganglio(Phi_total, DT)

    # 5. act_geom: orientación estable – misma variable
    Phi_total = aplicar_orientacion_por_gradiente(Phi_total, gradiente_E, DT)

    # 6. Actuación
    fL, fR, sf_v, asim, _ = calcular_parametros_actuacion(Phi_total)

    # 7. GED y eficiencia
    a_L = Phi_total[idx['aud_L'][0]:idx['aud_L'][1], :]
    a_R = Phi_total[idx['aud_R'][0]:idx['aud_R'][1], :]
    ged_L = calcular_ged_entre(region_int, a_L)

```

```

ged_R = calcular_ged_entre(region_int, a_R)
ged    = (ged_L + ged_R) / 2.0
efic, _ = calcular_eficiencia(Phi_total, ged)

historial_ef.append(efic)
if len(historial_ef) > TAU_EFICIENCIA * 2:
    historial_ef.pop(0)

explorador.actualizar(lf_prev, efic, fL, fR, sf_v)

# 8. Actualizar campo
Phi_total, Phi_vel_total, W_prof, W_rec, Phi_int_historia, \
    lf_activa, error_rec, _ = actualizar_campo(
        Phi_total, Phi_vel_total, W_prof, W_rec,
        Phi_int_historia, obj_L, obj_R, fL, fR, sf_v, DT
    )

# 9. Exploración activa
Phi_total = explorar_actuadores(Phi_total, explorador, lf_activa, efic, D
lf_prev    = lf_activa

# 10. Métricas
act_busc_val = calcular_senal_busqueda(Phi_total)
geom          = float(np.mean(np.tanh(
    Phi_total[idx['act_geom']][0]:idx['act_geom'][1], :])
))
G_act = float(np.mean(np.abs(
    Phi_total[idx['G']][0]:idx['G'][1], :])
))

for k, v in [
    ('ged_L',    ged_L), ('ged_R',    ged_R),
    ('grad_E',   gradiente_E), ('act_busc', act_busc_val),
    ('coh_rel',  coh_rel), ('geom',    geom),
    ('frac_L',   fL), ('frac_R',   fR),
    ('efic',     efic), ('lf',      lf_activa),
    ('w_rec',    np.mean(np.abs(W_rec))),
    ('w_prof',   np.mean(np.abs(W_prof))),
    ('G_act',    G_act),
]:
    hist[k].append(v)

if verbose and paso % 200 == 0:
    print(f"      t={paso*DT:.1f}s | GED={ged:.4f} | "
          f"gradE={gradiente_E:+.4f} | busc={act_busc_val:+.4f} | "
          f"coh={coh_rel:+.5f} | geom={geom:+.4f} | "
          f"G={G_act:.4f} | efic={efic:.3f} | ")

```

```

        f"LF={'ACTIVA' if lf_activa else 'inact'}")

def M(k): return float(np.mean(hist[k])) if hist[k] else 0.0
n_half = len(hist['geom']) // 2

geom_primera = float(np.mean(hist['geom'][:n_half])) if n_half > 0 else 0.0
geom_segunda = float(np.mean(hist['geom'][n_half:])) if n_half > 0 else 0.0
geom_conv     = (geom_primera * geom_segunda > 0) if n_half > 0 else False
busc_segunda = float(np.mean(hist['act_busc'][n_half:])) if n_half > 0 else 0
grad_medio   = M('grad_E')

if verbose:
    print(f"\n Resumen:")
    print(f"    GED L/R:                {M('ged_L'):.4f} / {M('ged_R'):.4f}")
    print(f"    Gradiente energético:    {grad_medio:+.4f}")
    print(f"    act_busc (2ª mitad):     {busc_segunda:+.4f}")
    print(f"    act_geom (2ª mitad):     {geom_segunda:+.4f}")
    print(f"    Convergencia geom:       {'✅ estable' if geom_conv else '⚠️ '})
    print(f"    Coherencia (diagnóstico): {M('coh_rel'):+.5f}")
    print(f"    Eficiencia media:        {M('efic'):.4f}")
    print(f"    LF activa (%):           {100*M('lf'):.1f}%")
    print(f"    Mejor efic explorada:    {explorador.mejor_eficiencia:.4f}")

return {
    'hist': hist,
    'geom_primera': geom_primera, 'geom_segunda': geom_segunda,
    'geom_conv': geom_conv, 'busc_segunda': busc_segunda,
    'grad_medio': grad_medio, 'coh_media': M('coh_rel'),
    'mejor_ef': explorador.mejor_eficiencia,
    'phi_total': Phi_total, 'phi_vel': Phi_vel_total,
    'W_prof': W_prof, 'W_rec': W_rec,
    'Phi_int_historia': Phi_int_historia,
}

# =====
# MAIN — PROTOCOLO CÍCLICO
# =====
def main():
    archivos = cargar_todos_binaurales('audio_binaural', 35.0)
    if not archivos:
        print("\nERROR: No se encontraron archivos. Ejecutar preprocesar_binaural")
        return

    Phi_total, Phi_vel_total, W_prof, W_rec, \
        Phi_int_historia, explorador = entrenar(archivos, 30.0)

```

```

FASES = [
    ("F2", 'voz_pos',          +60.0, 20.0, "Dominio - voz +60°"),
    ("F3", 'musica_pos',       +60.0, 20.0, "No entrenado - música +60°"),
    ("F4", 'tono_pos',         +60.0, 20.0, "No entrenado - tono +60°"),
    ("F5", 'ritmos_pos',       +60.0, 20.0, "Ritmos irregulares +60°"),
    ("F6", 'ondas_pos',        +60.0, 20.0, "Ondas mixtas +60°"),
    ("F7", 'pulso_pos',        +60.0, 20.0, "Pulso logarítmico +60°"),
    ("F8", 'viento_pos',       +60.0, 20.0, "Viento +60°"),
    ("F9", 'voz_viento1_pos',  +60.0, 20.0, "Degradado - voz+viento1 +60°"),
    ("F10", 'voz_viento2_pos', +60.0, 20.0, "Degradado - voz+viento2 +60°"),
    ("F11", 'ruido_pos',       +60.0, 20.0, "Perturbación basal - ruido +60°"),
    ("F12", 'bigbang_pos',     +60.0, 20.0, "BigBang +60° (20s por ciclo)"),
    ("F13", 'voz_neg',         -60.0, 20.0, "Re-acoplamiento opuesto - voz -60°")
]

```

```

N_CICLOS = 100

```

```

CICLOS_LOG = {1, 10, 25, 50, 75, 100}

```

```

# Punteros de posición en audio por clave - avanza entre ciclos

```

```

punteros = {clave: 0 for clave in archivos}

```

```

historial_ef = []

```

```

registro_ciclos = []

```

```

print()

```

```

print("=" * 100)

```

```

print("INICIANDO PROTOCOLO CICLICO - " + str(N_CICLOS) + " ciclos x 13 fases")

```

```

print("El campo persiste entre ciclos. Log detallado en ciclos: " + str(sorte))

```

```

print("=" * 100)

```

```

for ciclo in range(1, N_CICLOS + 1):

```

```

    verbose = ciclo in CICLOS_LOG

```

```

    if verbose:

```

```

        print()

```

```

        print("-" * 60)

```

```

        print("CICLO " + str(ciclo) + "/" + str(N_CICLOS))

```

```

        print("-" * 60)

```

```

    else:

```

```

        print(" Ciclo " + str(ciclo).rjust(3) + "...", end="", flush=True)

```

```

    metricas_ciclo = {}

```

```

    for fid, clave, angulo, dur, desc in FASES:

```

```

        if clave not in archivos:

```

```

            continue

```

```

        _, sr, c_L_full, c_R_full = archivos[clave]

```

```

n_pasos = int(dur / DT)
hop      = int(sr * HOP_FFT_MS / 1000)
vent     = int(sr * VENTANA_FFT_MS / 1000)

inicio_m = punteros[clave] * hop
# Reiniciar si el audio se agota
if inicio_m + n_pasos * hop > len(c_L_full):
    punteros[clave] = 0
    inicio_m        = 0

fin_m = inicio_m + n_pasos * hop + vent
c_L    = c_L_full[inicio_m : min(fin_m, len(c_L_full))]
c_R    = c_R_full[inicio_m : min(fin_m, len(c_R_full))]

needed = n_pasos * hop + vent
if len(c_L) < needed:
    c_L = np.pad(c_L, (0, needed - len(c_L)))
    c_R = np.pad(c_R, (0, needed - len(c_R)))

punteros[clave] += n_pasos

if verbose:
    print(" [" + fid + "] " + desc)

res = simular_fase(
    Phi_total, Phi_vel_total, W_prof, W_rec,
    Phi_int_historia, historial_ef, explorador,
    sr, c_L, c_R, dur, verbose=verbose
)

metricas_ciclo[fid] = res
Phi_total          = res['phi_total']
Phi_vel_total      = res['phi_vel']
W_prof             = res['W_prof']
W_rec              = res['W_rec']
Phi_int_historia   = res['Phi_int_historia']

# ---- Métricas del ciclo ----
def gc(fid, k):
    return metricas_ciclo[fid][k] if fid in metricas_ciclo else None

grad_f2 = gc('F2', 'grad_medio')
grad_f13 = gc('F13', 'grad_medio')
busc_f2 = gc('F2', 'busc_segunda')
busc_f13 = gc('F13', 'busc_segunda')
geom_f2 = gc('F2', 'geom_segunda')
geom_f13 = gc('F13', 'geom_segunda')

```

```

coh_f2    = gc('F2', 'coh_media')
coh_f13   = gc('F13', 'coh_media')
mejor_ef  = max([v.get('mejor_ef', 0.0)
                 for v in metricas_ciclo.values()
                 if isinstance(v, dict)], default=0.0)

c28 = (grad_f2  is not None and grad_f13 is not None
       and grad_f2 * grad_f13 < 0)
c21 = (busc_f2  is not None and busc_f13 is not None
       and busc_f2 * busc_f13 < 0)
c29 = (geom_f2  is not None and geom_f13 is not None
       and abs(geom_f2 - geom_f13) > 0.01)

registro_ciclos.append({
    'ciclo':    ciclo,
    'grad_f2':  grad_f2,  'grad_f13': grad_f13,
    'busc_f2':  busc_f2,  'busc_f13': busc_f13,
    'geom_f2':  geom_f2,  'geom_f13': geom_f13,
    'coh_f2':   coh_f2,   'coh_f13':  coh_f13,
    'mejor_ef': mejor_ef,
    'c28':      c28, 'c21': c21, 'c29': c29,
})

def fmt(v):
    return (str(round(v, 4)).rjust(8) if v is not None else "      N/A")

resumen = (
    "gradE F2=" + fmt(grad_f2) + " F13=" + fmt(grad_f13) + " | "
    "busc F2=" + fmt(busc_f2) + " F13=" + fmt(busc_f13) + " | "
    "geom F2=" + fmt(geom_f2) + " F13=" + fmt(geom_f13) + " | "
    "efic=" + str(round(mejor_ef, 4)) + " | "
    "C28=" + ("OK" if c28 else "--") + " "
    "C21=" + ("OK" if c21 else "--") + " "
    "C29=" + ("OK" if c29 else "--")
)

if verbose:
    print(" Ciclo " + str(ciclo).rjust(3) + " - " + resumen)
else:
    print(" " + resumen)

# ---- DIAGNÓSTICO FINAL ----
print()
print("=" * 100)
print("DIAGNOSTICO FINAL - v90 Protocolo ciclico 100x")
print("=" * 100)

```

```

n_c28 = sum(1 for r in registro_ciclos if r['c28'])
n_c21 = sum(1 for r in registro_ciclos if r['c21'])
n_c29 = sum(1 for r in registro_ciclos if r['c29'])
primer_c29 = next((r['ciclo'] for r in registro_ciclos if r['c29']), None)

vals_ini = [abs(r['geom_f2'] - r['geom_f13'])
             for r in registro_ciclos[:10]
             if r['geom_f2'] is not None and r['geom_f13'] is not None]
vals_fin = [abs(r['geom_f2'] - r['geom_f13'])
            for r in registro_ciclos[-10:]]
            if r['geom_f2'] is not None and r['geom_f13'] is not None]
dif_ini = float(np.mean(vals_ini)) if vals_ini else 0.0
dif_fin = float(np.mean(vals_fin)) if vals_fin else 0.0
ef_ini = float(np.mean([r['mejor_ef'] for r in registro_ciclos[:10]]))
ef_fin = float(np.mean([r['mejor_ef'] for r in registro_ciclos[-10:]]))

print()
print(" C28 (gradiente invierte): " + str(n_c28) + "/100 ciclos")
print(" C21 (act_busc invierte): " + str(n_c21) + "/100 ciclos")
print(" C29 (act_geom diferencial): " + str(n_c29) + "/100 ciclos")
print(" Primer ciclo con C29: " + (str(primer_c29) if primer_c29 else "ningu
print()
print(" Diferencia |geom_F2 - geom_F13|:")
print(" Ciclos 1-10: " + str(round(dif_ini, 4)))
print(" Ciclos 91-100: " + str(round(dif_fin, 4)))

if dif_fin > dif_ini * 1.5:
    tendencia = "MEJORA clara - la orientacion se consolida con el tiempo"
elif dif_fin > dif_ini * 1.1:
    tendencia = "leve mejora"
elif abs(dif_fin - dif_ini) < 0.002:
    tendencia = "estable - la orientacion persiste sin degradarse"
else:
    tendencia = "deterioro - revisar dinamica"
print(" Tendencia: " + tendencia)

print()
print(" Eficiencia explorada:")
print(" Ciclos 1-10: " + str(round(ef_ini, 4)))
print(" Ciclos 91-100: " + str(round(ef_fin, 4)))
print(" Tendencia: " + ("mejora" if ef_fin > ef_ini * 1.05 else "estable"))

# ---- CSV ciclos ----
with open('v90_ciclos.csv', 'w', newline='', encoding='utf-8') as f:
    wr = csv.writer(f)
    wr.writerow(['ciclo', 'grad_f2', 'grad_f13',
                 'busc_f2', 'busc_f13',

```

```

        'geom_f2', 'geom_f13',
        'coh_f2', 'coh_f13',
        'mejor_ef', 'c28', 'c21', 'c29'])
for r in registro_ciclos:
    wr.writerow([
        r['ciclo'],
        r['grad_f2'], r['grad_f13'],
        r['busc_f2'], r['busc_f13'],
        r['geom_f2'], r['geom_f13'],
        r['coh_f2'], r['coh_f13'],
        r['mejor_ef'],
        int(r['c28']), int(r['c21']), int(r['c29']),
    ])
print()
print(" CSV guardado: v90_ciclos.csv")

# ---- Gráfico ----
ciclos_x = [r['ciclo'] for r in registro_ciclos]
fig, axes = plt.subplots(2, 2, figsize=(16, 10))

axes[0, 0].plot(ciclos_x, [r['grad_f2'] for r in registro_ciclos],
                color='steelblue', lw=1.2, label='F2 +60')
axes[0, 0].plot(ciclos_x, [r['grad_f13'] for r in registro_ciclos],
                color='firebrick', lw=1.2, label='F13 -60')
axes[0, 0].axhline(0, color='k', lw=0.8, ls='--')
axes[0, 0].set_title('Gradiente energetico por ciclo')
axes[0, 0].set_xlabel('Ciclo')
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

axes[0, 1].plot(ciclos_x, [r['busc_f2'] for r in registro_ciclos],
                color='steelblue', lw=1.2, label='F2 +60')
axes[0, 1].plot(ciclos_x, [r['busc_f13'] for r in registro_ciclos],
                color='firebrick', lw=1.2, label='F13 -60')
axes[0, 1].axhline(0, color='k', lw=0.8, ls='--')
axes[0, 1].set_title('act_busc por ciclo')
axes[0, 1].set_xlabel('Ciclo')
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

axes[1, 0].plot(ciclos_x, [r['geom_f2'] for r in registro_ciclos],
                color='steelblue', lw=1.2, label='F2 +60')
axes[1, 0].plot(ciclos_x, [r['geom_f13'] for r in registro_ciclos],
                color='firebrick', lw=1.2, label='F13 -60')
axes[1, 0].axhline(0, color='k', lw=0.8, ls='--')
axes[1, 0].set_title('act_geom por ciclo')
axes[1, 0].set_xlabel('Ciclo')

```

```

axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)

# Marcar ciclos donde C29 se cumple
c29_ciclos = [r['ciclo'] for r in registro_ciclos if r['c29']]
for ax in axes.flat:
    for c in c29_ciclos:
        ax.axvline(c, color='green', alpha=0.12, lw=0.7)

axes[1, 1].plot(ciclos_x, [r['mejor_ef'] for r in registro_ciclos],
                color='darkorange', lw=1.2)
axes[1, 1].set_title('Mejor eficiencia explorada por ciclo')
axes[1, 1].set_xlabel('Ciclo')
axes[1, 1].grid(True, alpha=0.3)

plt.suptitle(
    'VSTCosmos v90 - 100 ciclos x 13 fases\n'
    'C29 en ' + str(n_c29) + '/100 ciclos'
    + (' (primer ciclo: ' + str(primer_c29) + ') ' if primer_c29 else ''),
    fontsize=12
)
plt.tight_layout()
plt.savefig('v90_evolucion.png', dpi=150)
print(" Grafico guardado: v90_evolucion.png")

print()
print("=" * 100)
print("EXPERIMENTO COMPLETADO")
print("=" * 100)

if __name__ == "__main__":
    main()

```